

Week16_성능 최적화

성능 최적화

데이터를 사용한 성능 최적화

: 데이터를 사용한 성능 최적화에서 가장 알려진 방법은 최대한 많은 데이터를 수집하는 것이다. 데이터 수집이 제한된 상황이라면, 임의로 데이터를 생성하는 방법도 있다.

- 최대한 많은 데이터 수집 : 일반적으로 ml,dl 알고리즘은 데이터 양이 많을수록 성능이 좋다.
- 데이터 생성하기: 데이터 수집이 한정된다면 데이터를 만들어 사용한다.
- 데이터 범위 조정하기: 사용하는 활성화 함수의 종류에 따라 데이터셋 범위를 조정
 - 시그모이드 활성화 함수: 0~1의 값
 - 하이퍼볼릭 탄젠트 활성화 함수: -1~1의 값

정규화, 규제화, 표준화도 성능 향상에 도움이 됨

알고리즘을 이용한 성능 최적화

수많은 알고리즘 중 최적의 알고리즘을 찾기 위해 다양한 알고리즘으로 모델을 훈련시켜 보고 최적의 성능을 보이는 알고리즘을 선택한다.

알고리즘 튜닝을 위한 성능 최적화

: 가장 핵심이 되는 성능 최적화 방법

- 진단: 성능 향상이 멈춘 원인을 진단하는 방법
 - 훈련 성능 > 검증 성능 → 과적합 → 규제화로 해결
 - 훈련&검증 성능이 둘 다 좋지 않다면 → 과소적합 → 네트워크 구조 변경, 에포크 수 조정
 - 훈련 성능이 검증을 넘어서는 변곡점이 존재 → 조기 종료
- 가중치: 오토인코더와 같은 비지도 학습으로 사전 훈련을 진행 후 지도 학습 진행
- 학습률: 네트워크 계층과 학습률을 비례하게 설정한다.
- 활성화 함수: 데이터 유형 및 데이터로 얻어내고자하는 결과에 따라 설정한다.
- 배치와 에포크: 큰 에포크와 작은 배치가 일반적이지만 다양한 테스트 진행 필요

- 옵티마이저 및 손실 함수: 확률적 경사 하강법을 많이 사용
- 네트워크 구성(네트워크 토폴로지)

양상블을 이용한 성능 최적화

- 양상블: 모델을 두 개 이상 섞어서 사용하는 것

하드웨어를 이용한 성능 최적화

CPU와 GPU 사용 차이

	CPU	GPU
구조	ALU, 컨트롤, 캐시	캐시는 적고, ALU는 많음
처리 방식	직렬 처리	병렬 처리
속도 및 성능	개별 코어 속도가 더 빠름	성능이 좋음
연산 분야	재귀 연산, 순차적 연산	역전파 미적분
활용 분야	파이썬, 매트랩	딥러닝

GPU를 이용한 성능 최적화

- CUDA: NVIDIA에서 개발한 GPU 개발 툴로 많은 양의 연산을 동시에 처리 가능하여 딥러닝, 채굴과 같은 수학적 계산에 많이 사용됨

GPU(CUDA) 설치 방법

1. GPU 장착 확인: 장치 관리자 → 디스플레이 어댑터에서 확인
2. CUDA 툴킷 설치: 드라이버 버전에 따른 CUDA 툴킷을 다운로드 받는다.
3. 시스템 변수 확인하기: CUDA_PATH 환경 변수 설정
4. cuDNN 설치
5. cmd에서 GPU 설치 확인

```
nvidia-smi
```

6. 파이토치 GPU 버전 설치하기

```
conda install pytorch torchvision torchaudio cudatoolkit=11.3 -c pytorch
```

하이퍼파라미터를 이용한 성능 최적화

: 배치 정규화, 드롭아웃, 조기 종료

배치 정규화



유사 용어 정리

정규화(스케일링)

: 사용자가 원하는 범위로 데이터 범위를 제한하는 것

ex) 0~255 사이의 픽셀값을 갖는 이미지 데이터를 255로 나눠 0~1.0 사이의 값으로 만들기

- MinMaxScaler() 기법으로 조정

규제화

: 모델 복잡도를 줄이기 위해 제약을 두는 방법

ex) 필터를 적용해 걸러진 데이터만 네트워크에 투입 → 빠르고 정확한 결과 습득 가능

- 드롭아웃, 조기 종료

표준화

: 기존 데이터를 평균 0, 표준편차가 1인 형태의 데이터로 만드는 방법

- 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용

배치 정규화: gradient vanishing/exploding 문제 해결을 위해 손실함수 사용, 초깃값 튜닝, 학습률 조정을 통해 데이터 분포를 안정시키고 학습 속도를 높인다.

- 기울기 소멸: 기울기가 0에 가까워져 학습 중지
- 기울기 폭발: 학습과정에서 기울기가 급격히 커지는 현상

→ 원인: 내부 공변량 변화

내부 공변량 변화란?

: 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상

→ 분산된 분포를 정규분포로 만들기 위해 미니 배치에 표준화와 유사한 방식 적용



배치 정규화 과정

1. 미니 배치 평균을 구한다.
2. 미니 배치의 분산과 표준편차를 구한다.
3. 정규화를 수행한다.
4. 데이터 분포를 조정한다.

장점: 데이터셋 분포가 일정해져 속도 향상의 효과가 있다.

단점:

- 배치 크기가 작을 때는 기존 값과 다른 방향으로 정규화가 진행될 수 있음
- RNN 같이 네트워크 계층별 적용이 필요한 모델은 너무 복잡해지고 비효율적일 수 있음

드롭아웃을 이용한 성능 최적화

드롭아웃: 훈련 시 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 대해서는 가중치 업데이트를 진행하지 않는 방법

- 매 단계마다 사용하지 않는 뉴런을 교체하며 훈련한다.
- 은닉층 배치 노드 중 일부를 임의로 끄면서 학습 → 지나친 학습 방지

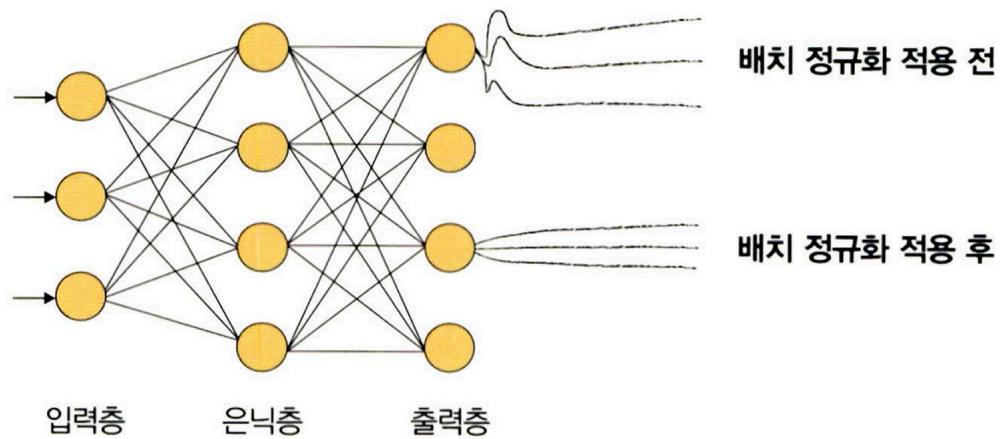
실습 내용 코랩 참조

```
# 배치 정규화가 포함된 네트워크
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.BatchNorm1d(48), # 배치 정규화 적용 부분
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )
```

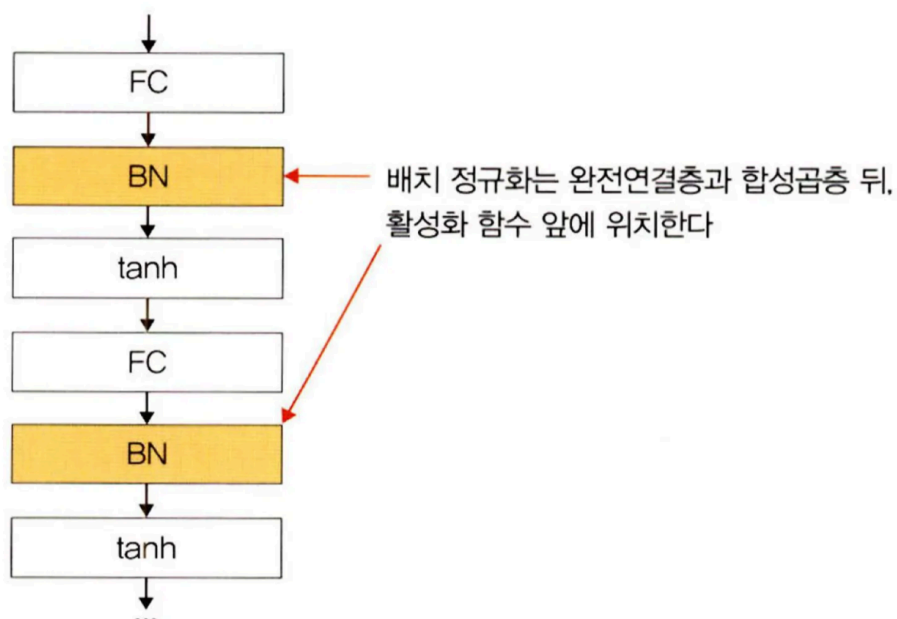
```
def forward(self, x):
    x = x.view(x.size(0), -1)
    x = self.classifier(x)
    return x
```

nn.BatchNorm1d(48) : 배치 정규화가 적용되는 핵심 코드

- 파라미터: 특성 개수로 이전 계층의 출력 채널이 됨
- 배치 정규화를 사용하는 이유: 신경망 층이 깊어질수록 엉뚱한 방향으로 학습이 진행되는 것을 방지하기 위해 배치 정규화를 적용해 입력 분포를 고르게 맞추어 준다.



- 배치 정규화의 위치



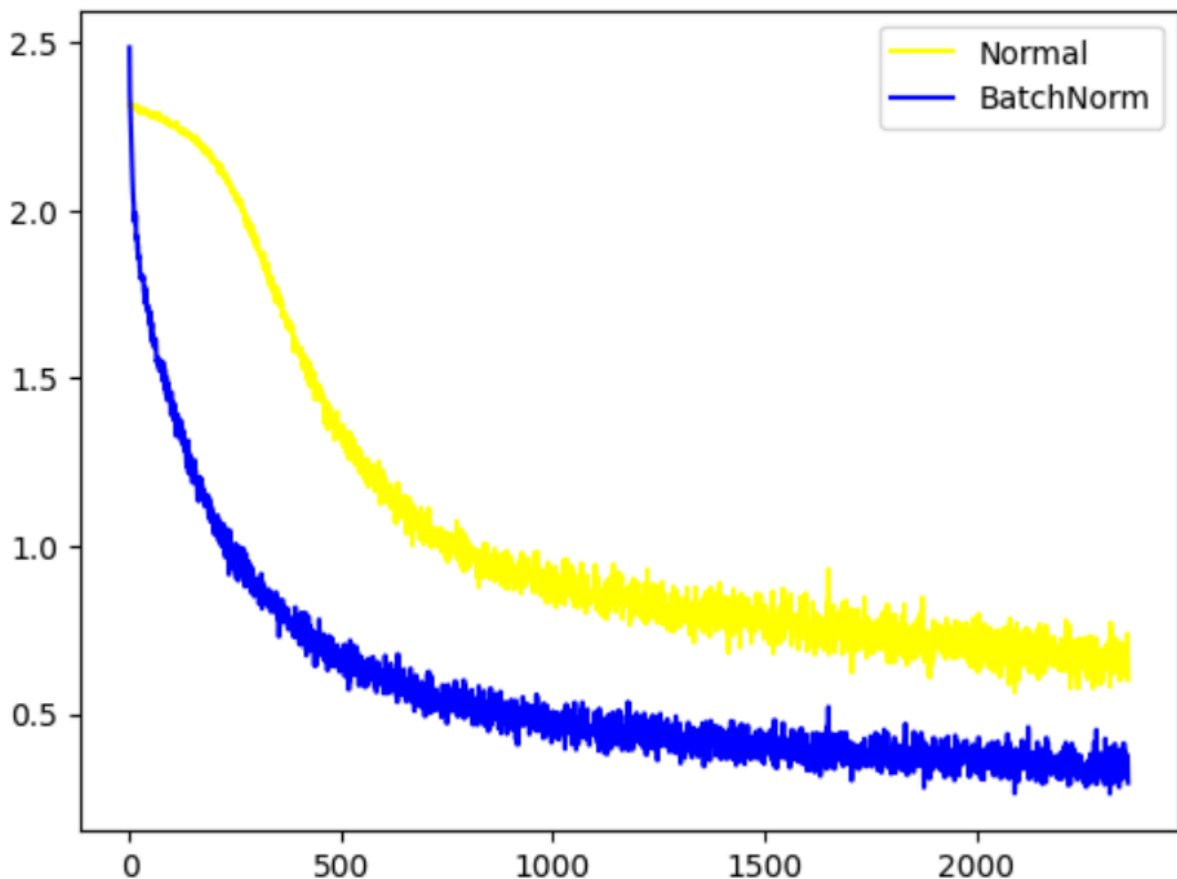
조기 종료를 이용한 성능 최적화

- 조기 종료: 뉴럴 네트워크가 과적합을 회피하는 규제 기법으로, 훈련 데이터와 별도로 검증 데이터를 준비하고 매 에포크마다 검증 데이터에 대한 오차를 측정하여 모델의 종료 시점을 제어한다.

| 실습 코드 및 설명 코랩 참조

모델 오차 그래프 살펴보기

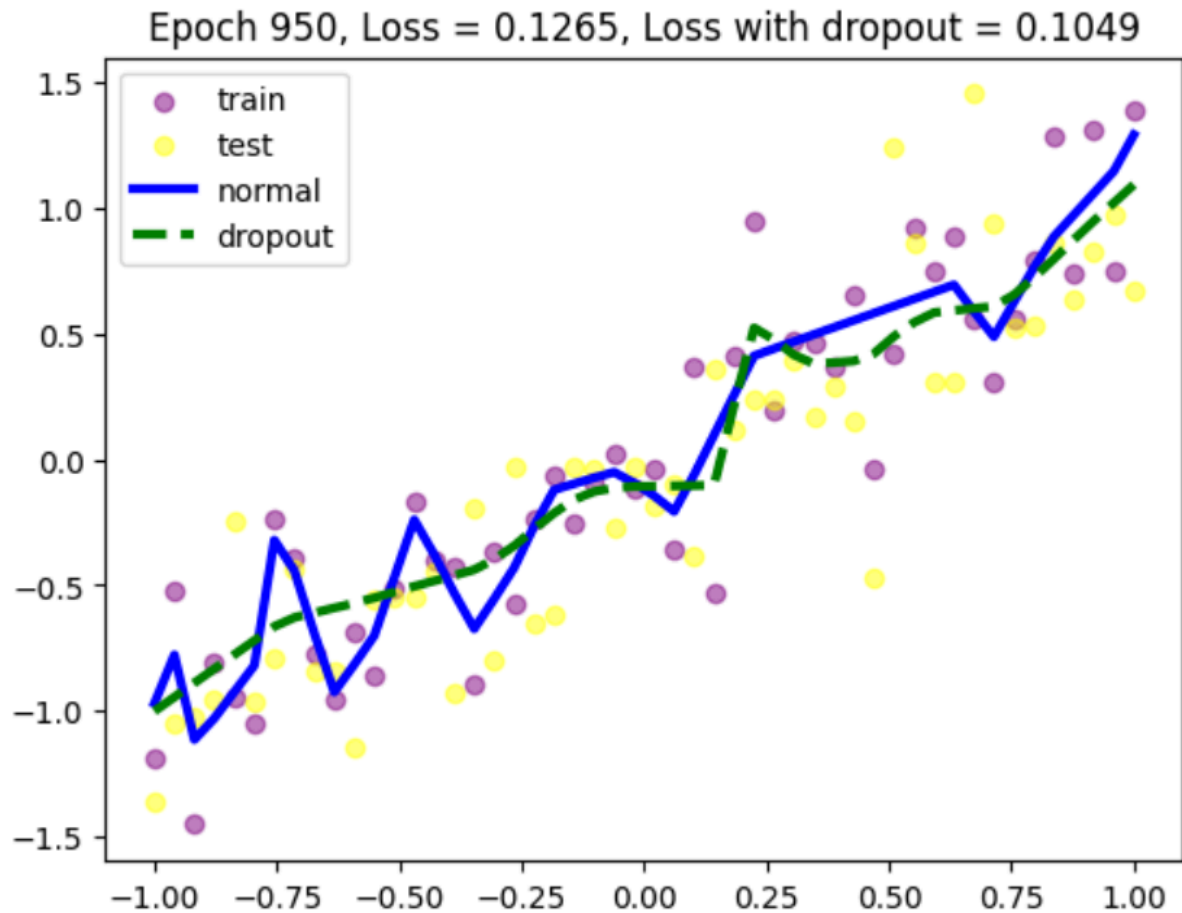
배치정규화



- 둘 다 시간이 흐를수록 오차는 줄어들지만, 배치정규화 적용 모델의 경우 더 낮은 값으로 안정적인 범위 내에서 오차가 줄어들고 있다.

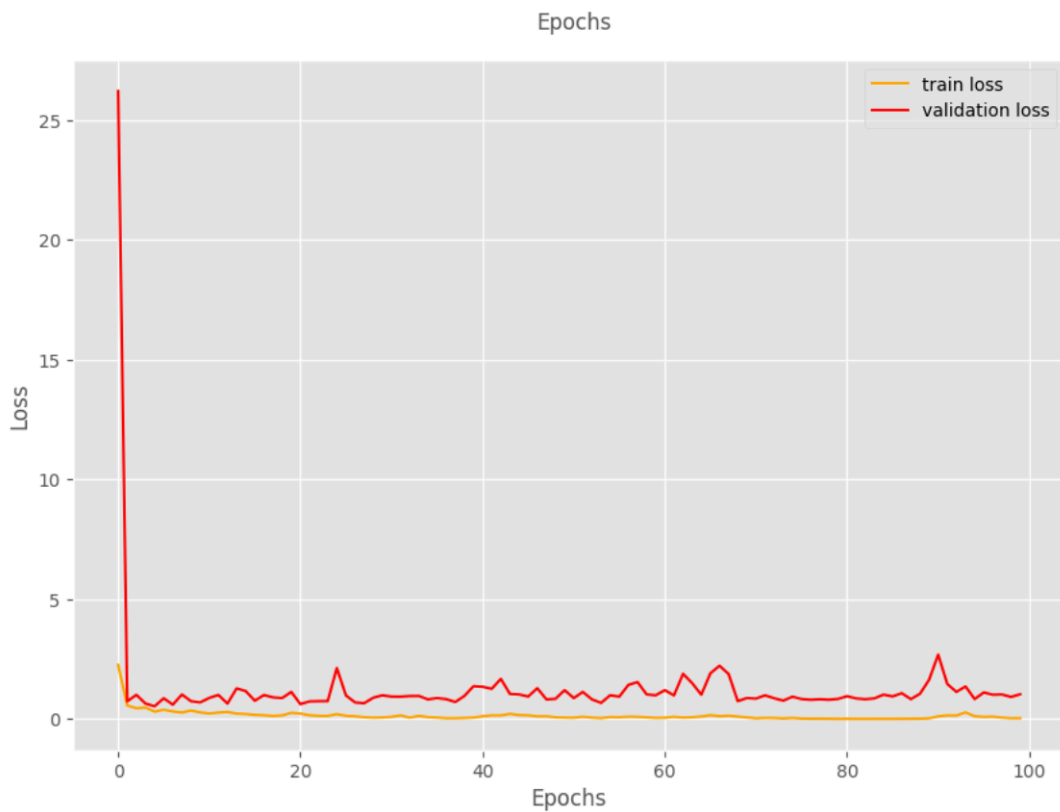
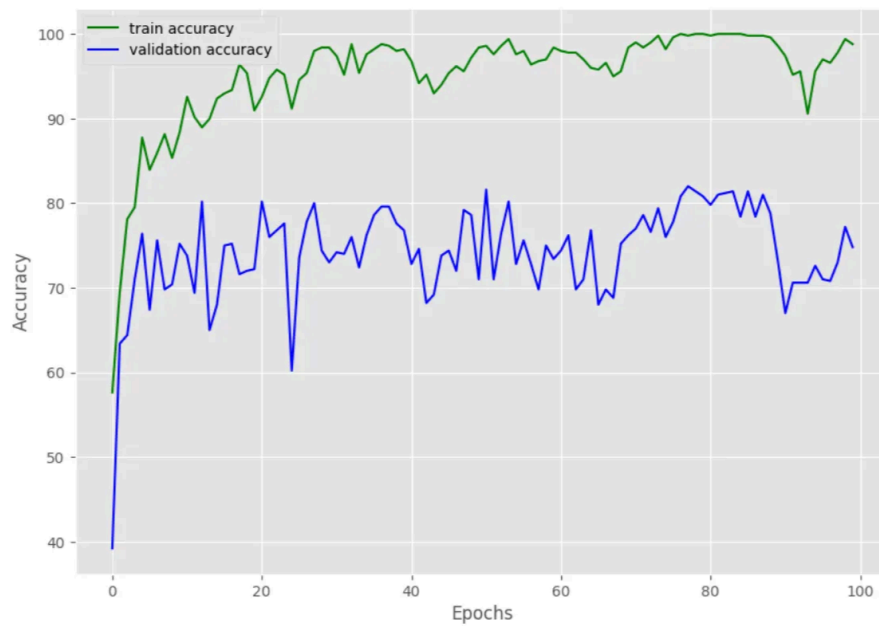
| 배치 정규화 적용 모델은 에포크가 진행될수록 오차 감소 & 안정적 학습을 하고 있다.

드롭아웃



- 드롭아웃 적용 시 오차가 더 낮다.
- 훈련횟수가 늘어날수록 일반 모델은 과적합 현상을 보이지만, 드롭아웃 적용 모델은 과적합이 발생하지 않는 걸 확인할 수 있다.

조기 종료



- 아무 인수도 적용하지 않았을 때는 정확도에 매우 많은 변동이 있고, 오차도 미세하게 우상향한다.
- 학습률 감소 적용 시에는 완만한 곡선 형태의 정확도와 오차도 우상향하는 현상이 사라졌다.

- 조기 종료 적용 시에는 검증 정확도 그래프가 불안정하지만 오차가 많이 낮아졌음을 확인할 수 있었다.

⇒ 결론적으로, 기존 출력 그래프를 해석하여 학습률 스케줄러를 이용한 학습률 조정 기법과 조기 종료와 같은 성능 기법을 적용할 방안을 고안하는 것이 중요하다.